DAY 9 | HANDS-ON ACTIVITIES

Node.js and Express: Problem Statements

Seven problems. Run Node.js scripts in Colab with !node file.js, or in the VS Code terminal. Test the API with Postman or a client script.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Modules Warm-up

Level: Warm-up | **Concepts:** module.exports, require, fs/promises

Create store.js that exports add(title), list() and save(path). save writes the tasks to a JSON file with fs/promises. Use it from main.js to add three tasks, save them and print the file contents.

Sample run / expected result

```
Added 3 tasks
Saved to tasks.json
[{"id":1,"title":"Read post-read"}, ...]
```

Hints

- const fs = require("fs/promises");
- await fs.writeFile(path, JSON.stringify(tasks, null, 2))

Problem 2: Read the Lockfile NEW (v4)

Level: Easy | **Concepts:** package.json ranges, package-lock.json

Install express in a new project. Compare the range in package.json with the exact version in package-lock.json. Write check_lock.js that prints every top-level dependency with its range and its locked version.

Sample run / expected result

```
express   range ^5.1.0   locked 5.1.0
nodemon   range ^3.1.0   locked 3.1.10 (dev)
```

Hints

- JSON.parse(fs.readFileSync("package-lock.json", "utf8")).packages["node_modules/express"].version

Problem 3: Audit and Report NEW (v4)

Level: Medium | **Concepts:** npm audit, severity, vulnerability report

Run npm audit --json on your project (or use the sample audit file from the notebook). Write a script that counts findings by severity and prints one row per package in the vulnerability report format: finding, tool, severity, affected package, proposed fix.

Sample run / expected result

```
critical 0 | high 1 | moderate 2 | low 0
V-01 | npm audit | high | example-pkg | upgrade to 2.4.1
```

Hints

- Exit with code 1 when high or critical findings exist, so CI can fail the build.

Problem 4: Express Tasks API

Level: Medium | **Concepts:** Express routes, express.json, status codes

Build GET /tasks, GET /tasks/:id, POST /tasks (400 without title, 201 on success), PUT /tasks/:id and DELETE /tasks/:id (204). Return 404 for unknown ids. Use a router and a controller file.

Sample run / expected result

```
POST /tasks {} -> 400 {"error": "title is required"}
DELETE /tasks/1 -> 204
```

Hints

- Number(req.params.id) converts the id.
- res.status(204).end()

Problem 5: Logging Middleware and 404s

Level: Easy | **Concepts:** app.use, next()

Add middleware that logs method, path, status and time taken for every request, and a catch-all 404 handler for unknown routes.

Sample run / expected result

```
POST /tasks 201 3ms
GET /nope 404 1ms
```

Hints

- res.on("finish", ...) runs after the response is sent.

Problem 6: Async Errors

Level: Challenging | **Concepts:** async routes, error middleware

Make the store async (every function returns a promise after a short delay). Add a route that throws when the store is "down". Show that the error middleware returns a clean 500 JSON response and that the server keeps running.

Sample run / expected result

```
GET /tasks?simulate=down -> 500 {"error": "Something went wrong"}
GET /health -> 200
```

Hints

- Error middleware has four arguments: (err, req, res, next).
- On Express 4 wrap handlers: const wrap = fn => (req, res, next) => fn(req, res, next).catch(next);

Problem 7: Postman Collection

Level: Stretch | **Concepts:** Postman, environments, tests

Create a collection with one request per endpoint using {{baseUrl}}. Add at least one test per request that checks the status code. Run the whole collection and export it.

Sample run / expected result

```
Collection run: 8 requests, 8 tests passed
```

Hints

- Import the collection generated in the notebook as a starting point.